

Stream: Design Space Exploration of Layer-Fused DNNs on Heterogeneous Dataflow Accelerators

Arne Symons , Linyan Mei , Steven Coleman , Pouya Houshmand , *Member, IEEE*, Sebastian Karl , and Marian Verhelst , *Fellow, IEEE*

Abstract—As the landscape of deep neural networks evolves, heterogeneous dataflow accelerators, in the form of multi-core architectures or chiplet-based designs, promise more flexibility and higher inference performance through scalability. So far, these systems exploit the increased parallelism by coarsely mapping a single layer at a time across cores, which incurs frequent costly off-chip memory accesses, or by pipelining batches of inputs, which falls short in meeting the demands of latency-critical applications. To alleviate these bottlenecks, this work explores a new fine-grain mapping paradigm, referred to as layer fusion, on heterogeneous dataflow accelerators through a novel design space exploration framework called *Stream*. *Stream* captures a wide variety of heterogeneous dataflow architectures and mapping granularities, and implements a memory and communication-aware latency and energy analysis validated with three distinct state-of-the-art hardware implementations. As such, it facilitates a holistic exploration of architecture and mapping, by strategically allocating the workload through constraint optimization. The findings demonstrate that the integration of layer fusion with heterogeneous dataflow accelerators yields up to $2.2\times$ lower energy-delay product in inference efficiency, addressing both energy consumption and latency concerns. The framework is available open-source at: github.com/kuleuven-micas/stream.

Index Terms—Deep neural networks, layer fusion, design space exploration, heterogeneous systems, dataflow, accelerators.

I. INTRODUCTION

DEEP neural networks (DNNs) have significantly advanced various fields, including computer vision, natural language processing, and signal analysis. However, this progress has come with a substantial increase in the size of networks (weights) and intermediate data values (activations). Simultaneously, the demand for edge processing has risen, imposing strict energy and latency requirements with a constraint on memory footprint.

Received 12 April 2024; revised 30 September 2024; accepted 6 October 2024. Date of publication 10 October 2024; date of current version 12 December 2024. This work was supported in part by the European Research Council (ERC) under Grant Agreement 101088865, in part by the European Union's Horizon 2020 programme under Grant Agreement 101070374, in part by the Flanders AI Research Program, and in part by the KU Leuven. Recommended for acceptance by P. Sainrat. (*Corresponding author: Arne Symons.*)

The authors are with the Department of Electrical Engineering (ESAT), KU Leuven, 3001 Leuven, Belgium (e-mail: arne.symons@kuleuven.be).
Digital Object Identifier 10.1109/TC.2024.3477938

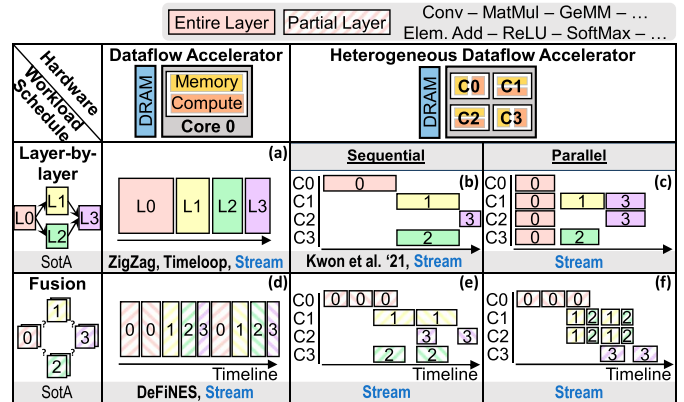


Fig. 1. A conceptual example showing different ways of scheduling a deep neural network workload onto different hardware accelerators.

A. Dataflow Accelerator Architectures

Dataflow accelerator (DA) architectures have evolved from single-core designs with specific dataflows [1], [2], [3] to multi-core and chiplet-based systems [4], [5], [6], [7], [8], increasing computing parallelism. Heterogeneous dataflow accelerator (HDA) architectures offer specialized processing for diverse DNN layer types [9], [10], [11], employing varied dataflows, PE array sizes, and memory capacities. Fig. 1 illustrates DNN inference scheduling on DA and quad-core HDA architectures. Fig. 1(a)–1(c) shows *layer-by-layer* or *layer-first* scheduling, which can improve throughput through pipelining but fails to enhance latency due to under-utilized cores (Fig. 1(b)).

B. Fine-Grained Scheduling Strategies

Parallelizing a layer across multiple cores for increased core utilization introduces challenges for HDA architectures, as shown in Fig. 1(c). An alternative approach is *layer-fused* scheduling [12], also known as *depth-first* [13] or *cascaded* [14] scheduling (Fig. 1(d)–1(f)). This technique processes a segment of outputs sequentially through a fused stack of layers, reducing memory footprint and off-chip data accesses while enhancing workload parallelism (Fig. 1(e)). Combined with layer parallelization (Fig. 1(f)), this improves core utilization but increases core-to-core transfers. Existing layer-fused scheduling techniques are often hardware-specific [10], [15], [16], [17], limiting performance evaluation across diverse scenarios. The

mapping of layer-fused DNNs onto various HDA architectures remains largely unexplored.

C. Contributions of This Work

This work introduces the first comprehensive exploration framework for HDA architectures, featuring fine-grained scheduling of layer-fused DNNs. The primary contributions are as follows:

- **Stream:** An open-source design space exploration framework called *Stream* is presented for the mapping of layer-fused DNNs on heterogeneous dataflow accelerator architectures. Stream offers a flexible mapping granularity and architecture representation, facilitating adaptable and comprehensive exploration across diverse dataflow strategies and architectural configurations through a rapid fine-grain data dependency extraction (Section III).
- **COALA:** The hardware performance of fine-grain mappings onto these architectures is estimated through a memory- and communication-aware latency analysis (COALA). The accuracy of COALA is validated through comparison with three state-of-the-art implementations of hardware accelerators architectures that employ layer fusion (Section IV).
- **WACO:** The allocation and scheduling of the fine-grain workload is optimized through constraint optimization (WACO). WACO achieves up to 32% better Energy-Delay Product (EDP) compared to a SotA genetic algorithm-based allocator due to its ability to partition layer parts across cores. Further experiments reveal up to a $2.2\times$ EDP reduction compared to traditional layer-by-layer scheduling and show Stream's ability to evaluate a wide range of HDA architectures (Sections V & VI).

II. BACKGROUND & RELATED WORKS

In this section, we first provide the background on DNN acceleration using dataflow accelerator architectures. Next, we detail mapping techniques of DNNs onto these architectures. The state-of-the-art comparison is summarized in Table I.

A. Dataflow Accelerator Architectures

Traditional accelerator cores, as shown in Fig. 2(b), utilize arrays of spatially unrolled processing elements (PEs) to enable parallel computations. Depending on the dataflow, different tiles of the DNN are parallellized on the PE array. To further increase compute parallelism while maintaining efficiency, the field has evolved towards multi-core or multi-chiplet-based architectures (Fig. 2(a)). This trend began with homogeneous multi-core systems, which replicate the same accelerator across multiple units. Examples include Planaria [4] with its partitionable systolic array, Illusion [6]'s 8-chip design with minimal on-chip memories, Simba [5]'s 36-chiplet architecture, and the prototype of a 4×4 clustered analog in-memory compute (AiMC) system [7].

Building on this foundation, heterogeneous dataflow accelerators (HDA) designs, such as those explored in [9] and implemented in DIANA [10], incorporate multiple sub-accelerators

TABLE I
COMPARISON OF STATE-OF-THE-ART FRAMEWORKS

Framework	HDA Model	Layer Fusion	Optimization Method	Perf. Evaluation
Kwon et al. [9]	✓	✗	Utilization Heuristic	Analytical Model
TVM [14]	✗	✓	Memory Heuristic	ARM Ethos-U
DeFiNES [18]	✗	✓	Exhaustive Tile Search	Analytical Model
DNNVM [19]	✗	✓	Subgraph Isomorphism	FPGA
Olympus [20]	✗	✓	X-partition, Dynamic Progr.	Interstellar
TileFlow [21]	✗	✓	Tree-based Analysis	Analytical Model
This Work (Stream)	✓	✓	Constraint Optimization	Analytical Model

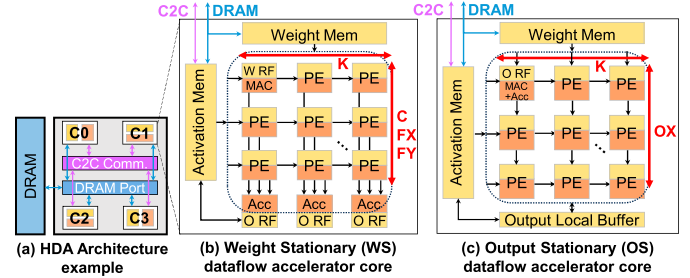


Fig. 2. Architectural schematics of dataflow accelerators. (a) Heterogeneous dataflow accelerator (HDA). (b) Weight stationary (WS) core. (c) Output stationary (OS) core.

(i.e. cores) with varying dataflow specializations. By combining different core types (like digital and AiMC cores) and dataflows, HDAs offer more tailored and efficient processing for a wider range of DNN layer types, addressing the growing diversity in neural network architectures.

B. Workload Allocation, Scheduling & Mapping

The recent trend towards multi-core accelerator systems enlarges the traditional mapping space. It is divided into three categories:

1) *Allocation:* Each layer of the workload must firstly be allocated to one or more cores (from now on referred to as *workload allocation*). Allocating a modern DNN of 50 layers onto e.g. a quad-core architecture yields 10^{30} possible workload allocations. Kwon et al. [9] allocate the workload to the heterogeneous cores through a utilization-based heuristic, but their work assumes a fixed communication topology through a shared buffer, which can incur unnecessary energy and latency overhead for small tensors.

2) *Scheduling:* Each allocated layer must subsequently be scheduled in time. This determines the execution order of

the layers (or their fine-grained tiles). In traditional layer-by-layer processing, the only scheduling flexibility comes from the possibility to reorder branches in the DNN. More scheduling opportunities, however, arise when scheduling at a sub-layer granularity. Instead of processing an entire layer at once, a smaller tile of each layer is processed and its outputs are immediately consumed to process tiles of the subsequent layers. The size of such a layer tile determines the *scheduling granularity*.

The scheduling of layer-fused DNNs presents its own set of challenges and opportunities. Challenges include diminished data reuse opportunities as layer tiles become smaller, together with a large increase in the number of valid scheduling orderings due to the diversity of the workload allocation on the HDA architectures. Despite these challenges, a careful mapping of layer-fused DNNs on HDA architectures can drastically improve the available workload parallelism and the retention of more data on-chip, thanks to the reduced data requirements per layer tile.

TVM [14] includes a cascading scheduler that explores different scheduling granularities, but is tied to the ARM Ethos-U microcontroller platform. DeFiNES [18] introduces an analytical cost model to enable fast depth-first scheduling design space exploration for single-core DA architectures. Yet, the exploration of layer fusion on HDA architectures remains largely unexplored. DNNVM [19] optimizes the scheduling through a number of heterogeneous compiler optimizations for FPGA-accelerated inference. Olympus [20] reaches memory-optimality for a single-core FPGA-based accelerator through layer fusion. In TileFlow [21], a tree-based representation of nested for loops achieves optimal layer fusion for a homogeneous multi-core accelerator architecture using shared memories.

3) *Mapping*: Lastly, the computations of each layer tile must be efficiently mapped onto each core, respecting the constraints stemming from the supported dataflow and memory resources of each individual core. The spatial dataflow plays a key role in the core's utilization, as a mismatch between the core's dataflow dimensions and the tile granularity will cause spatial under-utilization of the PE array. The temporal mapping, on the other hand, impacts the temporal data reuse from the different memories of the core's memory hierarchy [22], [23]. Many design space exploration (DSE) frameworks have arisen [24], [25], [26], [27], [28], [29], [30] to analytically estimate the HW cost and optimize the efficiency of the spatial and temporal mapping through loop optimizations such as unrolling, tiling and re-ordering. These frameworks model a layer-by-layer scheduling, summing up the energy and latency cost across layers.

III. STREAM FRAMEWORK

MOTIVATION & OVERVIEW

The increasing complexity of HDA architectures, characterized by the number of cores, each core's dataflow, memory distribution, and inter-core connection topology and bandwidth, significantly impacts hardware performance. Concurrently, the

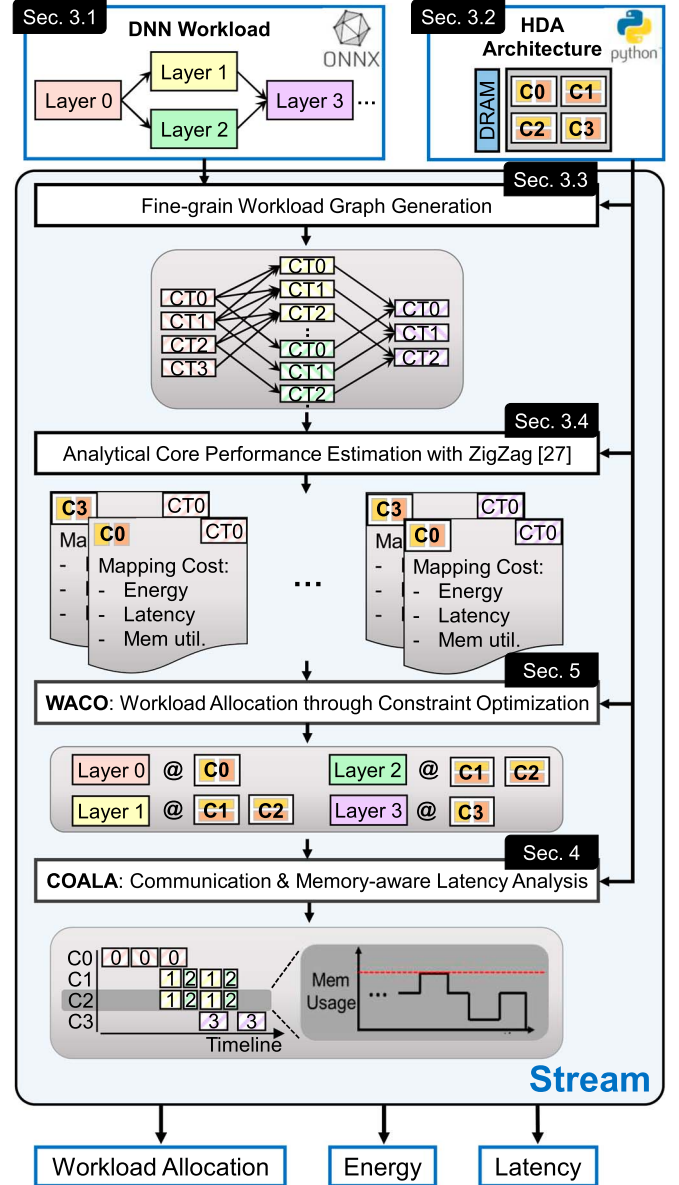


Fig. 3. Overview of the stream framework.

scheduling of layer-fused DNNs presents its own aforementioned challenges and opportunities.

To date, no framework exists that facilitates this co-exploration, hindered by the difficulty in rapidly extracting the fine-grain data dependencies between layer-fused tiles, accurately modeling constraints and overheads from the multi-core hardware architectures, and efficiently exploring the vast workload allocation space. Moreover, it is crucial that the framework is easily extendable to a wide variety of future architectures and scheduling paradigms. This motivates the need for a novel design space exploration framework: *Stream*. Stream aims to leverage the parallelization opportunities and address the challenges posed by both HDA architectures and fine-grain layer fusion, with accurate hardware performance modelling.

Fig. 3 shows an overview of Stream. The DNN workload is first converted into a fine-grain workload graph of layer

tiles based on the desired granularity and core dataflows. Next, the performance of each unique tile-core combination is estimated using an analytical single-core DSE framework, cfr. Section II-B. Given a workload allocation, which is optimized in the WACOengine, COALA estimates the latency and energy consumption of the DNN inference onto the HDA architecture, taking into account data communication and limited on-chip memory.

IMPLEMENTATION

A. Input: ONNX Workload

Stream converts each operator of the provided ONNX [31] model into a layer containing a set of nested for-loops of different dimensions and loop bounds. Each layer can contain a stride in any dimension and as a consequence Stream is capable of representing all popular DNN operators: Conv1d, Conv2d, Conv3d, MatMul, GeMM, Pooling, Softmax, Add, etc. The layers are interconnected in a directed acyclic graph.

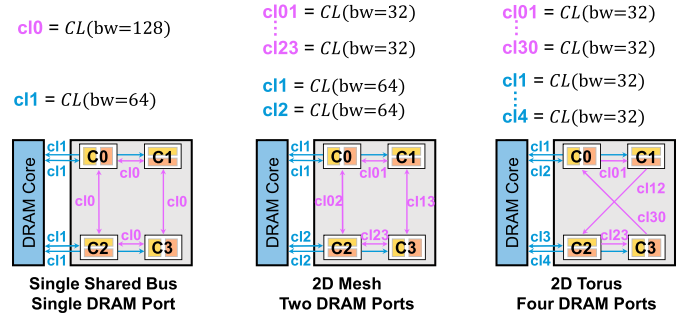
B. Input: HDA Architecture

The HDA architecture definition within Stream is also designed with versatility and adaptability at its core. A core is characterized by a PE array shape and a hierarchical memory structure. Each memory is defined with a specified capacity, a designated number of read, write, or read/write ports, and read/write energy costs. The ports are constrained by a limited bandwidth to simulate realistic hardware limitations accurately.

The cores in the HDA are interconnected in a directed graph $G = (V, E)$, where V is the set of vertices representing the cores, and E is the set of directed edges representing the communication links between these cores. Each edge $e \in E$ has an associated Communication Link (CL), which embodies the communication channel between cores. These CLs are characterized by a limited bandwidth, which mirrors the communication constraints found in actual hardware designs. This graph-based representation of core interconnectivity allows the HDA architecture to effectively emulate a wide range of modern core-to-core and off-chip communication topologies. Fig. 4 illustrates the flexibility using three examples ranging from a simple single shared bus architecture to sophisticated 2D mesh and torus configurations with varying off-chip communication ports.

C. Fine-Grain Workload Graph Generation

1) *Layer Fusion Approach*: Layer fusion in DNN models can be implemented through different strategies, such as integrating a primary operation with an auxiliary element-wise function (e.g., combining a convolutional layer with ReLU) or merging multiple core operations (e.g., consecutive convolutional layers). The Stream framework combines both, as its goal is to explore the possibilities of deep fusion, where tiles of layer activations are reused in a depth-first manner through the workload. This is achieved through the combination of partitioning each layer of the workload into finer computation tiles (CT) and inferring the dependencies between them. Moreover, the



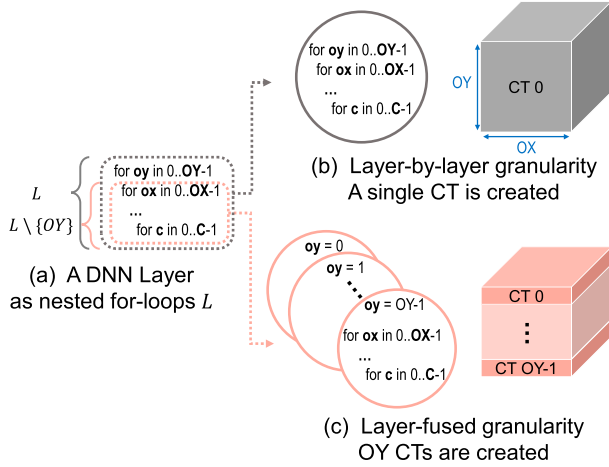


Fig. 5. Each layer is partitioned into one or more computation tiles (CT) depending on the desired granularity.

3) *Fine-Grain Data Dependencies*: After the identification of the CTs of each layer, the data dependencies between all CTs must be generated in order to schedule them correctly and with structured memory accesses. This process is split into two parts:

Intra-layer: First, the intra-layer CT dependency edges are inserted based on the outer-CT loop order. This approach serves several purposes: it ensures that the required data accesses of CTs within a layer are structured and hardware-friendly, as enforcing a consistent execution order avoids unstructured memory access patterns; moreover, it reduces redundant degrees of freedom in the scheduling process, streamlining the overall design while maintaining optimality, as the core already executes the CTs of a layer sequentially; and it maintains consistent scheduling across dimensions (e.g., the batch dimension) for all layers, ensuring regularity in execution and efficiency in fusion.

Inter-layer: Inter-layer CT dependencies are determined by identifying overlaps in data generated by CTs of one layer and consumed by the next. Given the fine-grained scheduling in modern DNNs, with up to 10^6 CTs, a direct pairwise check for dependencies would require 10^{12} operations, which is impractical. Instead, an efficient *R-tree* [32] algorithm is used to quickly identify these dependencies. Fig. 6 illustrates this process. First, an R-tree is constructed for the consumer layer's CTs (①). Then, the R-tree is queried with each producer CT to find overlapping consumer CTs (②). Although the example shown is simplified with 4 non-overlapping CTs, the method supports multi-dimensional overlapping ranges.

This R-tree-based approach greatly speeds up a baseline pairwise algorithm by three orders of magnitude, reducing dependency generation time from over 9 hours to 6 seconds.

D. Analytical Core Performance Estimation

In this step latency performance, energy consumption and memory requirements of executing individual CTs on each of the available HDA cores are extracted. As discussed in Section II-B, multiple DSE frameworks already exist for optimizing

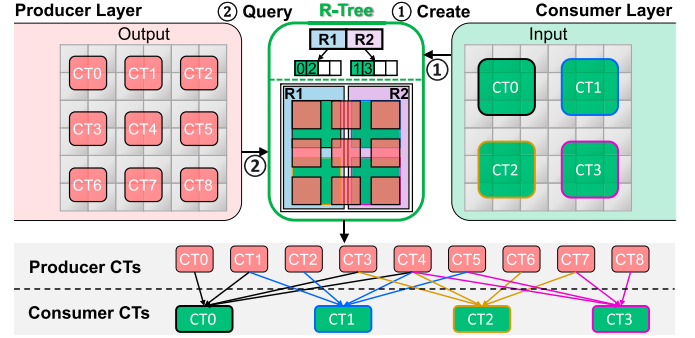


Fig. 6. Inter-layer CT dependency generation example using R-trees [32].

the mapping of layer-by-layer workloads onto single-core HW architectures [24], [26], [27], [28], [29], [30]. These frameworks model the hardware performance of a layer and optimize the spatial dataflow and temporal mapping for the targeted accelerator, in the form of loop ordering and tiling in the memory hierarchy.

Stream exploits ZigZag [27] to obtain the optimal intra-core mapping for each CT on each available core along with the associated hardware costs. This is achieved by parsing the for loops embedded within a CT (Fig. 5(c)) into the layer representation of ZigZag. If not all data can be tiled on-chip, the off-chip memory instance is added to the core's hierarchy. ZigZag provides a detailed latency model for both the on- and off-loading of data to/from off-chip memory. Moreover, it models data stalls resulting from inadequate memory bandwidth within the core [33]. The spatial dataflow and temporal mapping are optimized for latency [23].

Although this work integrates with ZigZag for core performance estimation, the flexible HDA template allows for the easy use of alternative performance estimation frameworks.

E. Workload Allocation

To effectively schedule the workload across time, each CT needs to be allocated to a core. This is essential for accurately evaluating the potential parallelism and the overheads associated with off-chip and core-to-core data transfers. For now, the allocation is considered given. Section V details how Stream achieves optimal workload allocation.

IV. COMMUNICATION & MEMORY-AWARE LATENCY ANALYSIS WITH COALA

The goal of the communication & memory-aware latency analysis (COALA) is to estimate the latency and energy consumption of a given workload, core performances, allocation and HDA, as illustrated in Fig. 7. Performing accurate estimations comes with two major challenges: 1) correctly assessing the core-to-core communication overhead given the HDA connectivity topology, and 2) taking into account the required amount of off-chip traffic for varying CT granularity and on-chip memory capacity. These overheads are modelled through the help of a communication manager and a memory manager.

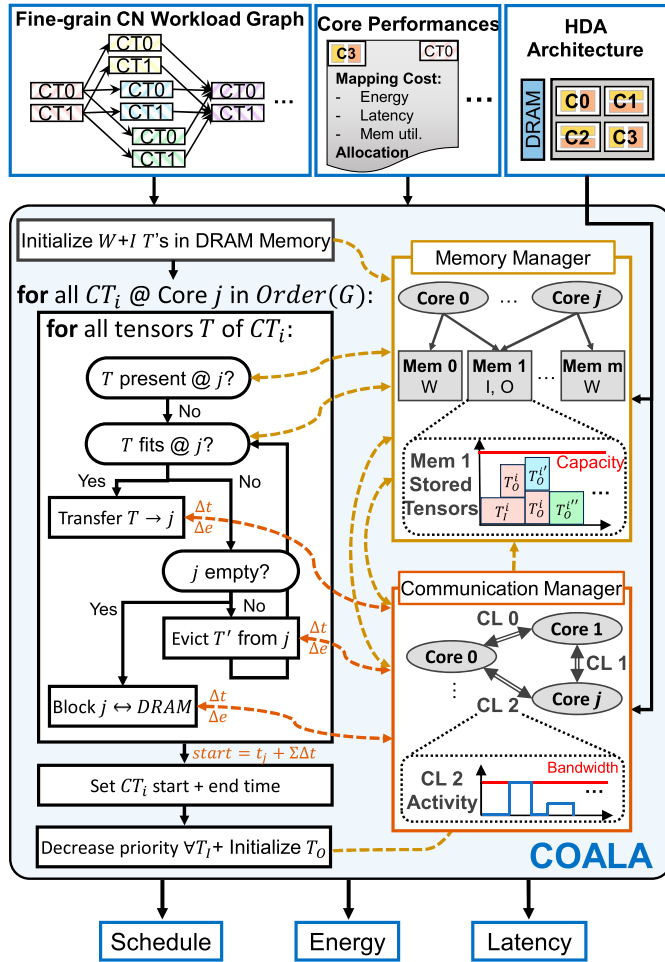


Fig. 7. Accurate scheduling is achieved through a communication & memory manager that take core-to-core and off-chip transfers, and limited on-chip memory into account.

A. Communication Manager

Whereas modeling the communication cost of transferring activations from core to core is relatively simple for traditional layer-by-layer execution [6], the complexity grows for finer CTs that can execute in parallel. To model transfer contention, a communication manager registers each transfer activity to CommunicationLink (CL) object(s) attached to all communication channels between the cores involved in the specific data transfer (Fig. 2). The transfer time Δt depends on the packet size and link bandwidth, and the energy overhead Δe depends on the unit energy cost. For multi-hop transfers, the communication manager finds the earliest time window in which all involved links are free and energies are accumulated.

B. Memory Manager

To know which tensor transfers are needed, a dedicated memory manager tracks the lifetime of tensors in each memory throughout the scheduling process. This allows to schedule memory eviction and reloading operations while ensuring the capacity of any of the on-chip memories is never surpassed.

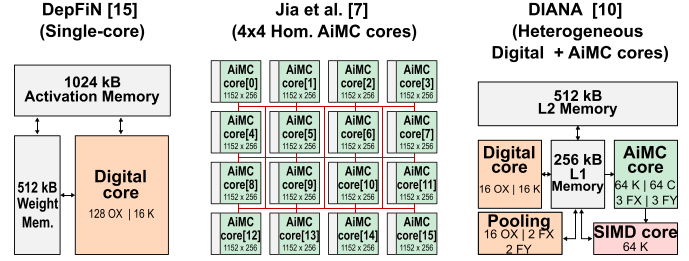


Fig. 8. Hardware architecture targets for the validation.

This work operates under the premise that data transfers always occur between the topmost memory level within each core.

As depicted in Fig. 7, the decision-making process for scheduling CT_i onto its allocated core j ($a_{i,j}$) as such involves several evaluations and actions by the memory manager. The CT requires the presence of one or more input tensors T_I and weight tensors T_W before it can be executed. Should the tensor T already reside in the on-chip memory of core j , no memory or communication overhead is incurred. Otherwise, the available memory space to store T is assessed. In case of sufficient space, the communication manager transfers T , updating the appropriate CommunicationLink (CL) activities and memories. Insufficient space triggers the eviction of one or more low-priority tensors T' stored in the core's memory. This priority T' is stored in the memory manager and computed based on the number of unscheduled CTs that require T' and its size. This approach evicts smaller tensors first to avoid the significant impact of off-loading large tensors, aiming to optimize memory usage and scheduling efficiency. Finally, in case all tensors have been evicted and there is still insufficient space, DRAM is included as the top-level memory for the core performance estimation, and an activity is registered for the link(s) between j and DRAM during the execution of CT_i for the fetching of smaller tiles of T .

Once all input tensors T_I required for CT_i are present, the CT is scheduled at available time t_j of core j plus the potential delays due to transfers $\Sigma \Delta t$. The priority of the input tensors is decreased, and the output tensor T_O is initialized in the memory manager.

C. Validation

Stream is deployed to model the behavior of three SotA taped-out HW architectures, in order to demonstrate its modeling flexibility and to quantify its COALALatency modeling and memory footprint accuracy, compared to that of their measured system performance.

HW Targets. Fig. 8 shows the three architectures considered in this validation. They are diverse in their core and memory architectures and supported scheduling granularities:

- 1) DepFiN [15] is a single core DNN accelerator designed for high-resolution pixel processing workloads which deploys layer fusion with line-based CTs. The architecture includes a digital core with an $OX|K$ dataflow, 512 kB of weight memory and 1 MB of activation memory. Off-chip communication is limited to 64 bit per clock cycle.

- 2) Jia et al.'s [7] multi-core architecture consists of a 4×4 array of analog in-memory-compute (AiMC) cores, enabling high throughput and energy efficiency through pipelined execution. The cores are grouped and communicate with a 128 bits per clock cycle on-chip network. The cores operate on a weight-stationary dataflow, with the weights loaded into the analog array before the computations. The weight loading can happen in parallel, through multiple off-chip bandwidth links of 128 bits per clock cycle.
- 3) DIANA [10], a heterogeneous multi-core AiMC + digital hybrid DNN accelerator SoC targets efficient end-to-end inference for edge applications. The architecture includes a shared 256 kB L1 memory to efficiently transfer activations between cores. The digital core employs an $OX|K$ dataflow, while the AiMC core includes, similarly to Jia et al.'s cores, a weight stationary dataflow with the weights loaded into the array. The SIMD core handles non-linearities and element-wise additions.

Each architecture's specification is modelled in Stream through the: a.) intra-core characteristics for each core (operand precision; PE array size; supported dataflow; memory hierarchy) and b.) inter-core characteristics (inter-core communication protocol: bandwidth-limited uni/bi-directional links going from one core's memory to another's or through a shared memory; inter-core connection topology; limited bandwidth off-chip link(s)).

Workload & Performance Targets. Each HW target has reported the measurements of their accelerator performance for different DNNs:

- 1) The latency performance and memory footprint of FSR-CNN [34], a fast super-resolution CNN, was measured on the single-core DepFiN processor.
- 2) Jia et al. measured the latency performance of a ResNet50 segment.
- 3) The latency performance and memory footprint of a ResNet18 segment was measured on the DIANA chip.

Each measured DNN is modelled in Stream at the scheduling granularity supported by the hardware. The intra-core mapping and workload allocation is fixed in accordance with the reported measurements.

Modeling Modes. To demonstrate the importance of different modeling aspects, the validation is performed without and with communication modeling:

- 1) A simple *intra-core* model which takes into account the core mapping performances of all CTs, which have potential temporal stalls because of insufficient bandwidth of the core memories.
- 2) The detailed *Stream* system model which uses COALAt model the core-to-core communication and off-chip accesses overhead. Furthermore, it facilitates detailed analysis of the memory footprint, offering valuable insights for expanded exploration of the design space.

The outcomes of the validation process are presented in Table II and Fig. 9 visualizes the schedule and L1 memory usage of the DIANA results. Latency is characterized by the

TABLE II
VALIDATION RESULTS FOR TARGETED ARCHITECTURES

Architecture	Latency Validation (10^4 clock cycles)				
	Measured	Modelled			
		Intra-core		Stream	
DepFiN [15]	618	502	81%	637	96%
Jia et al. [7]	6.6	6.2	93%	6.4	97%
DIANA [10]	81.2	78.1	95%	79.2	97%
	Memory Validation (KB)				
		Intra-core		Stream	
DepFiN [15]	238	×	×	262	90%
Jia et al. [7]	N/A	×	×	36	N/A
DIANA [10]	134	×	×	114	85%

completion time of the final computation or communication task, measured in cycles. Concurrently, the memory footprint is determined by the peak memory utilization across the scheduling period, predicated on the assumption of 8-bit quantization for both activations and weights. Notably, Stream demonstrates superior precision in latency modeling these diverse architectures compared to the intra-core model, which neglects the overhead and contention for core-to-core and off-chip accesses.

V. WORKLOAD ALLOCATION AND SCHEDULING OPTIMIZATION WITH WACO

Goal

The allocation and scheduling of partitioned workloads across available cores is a critical challenge for efficiently deploying DNNs on HDA architectures. Cores with different dataflows can be efficient for some DNN kernels but under-utilized for others. Besides allocation, tiles must be scheduled correctly, accounting for fine-grain CT dependencies. Fine-grain layer fusion can produce large workload graphs with up to 10^6 tiles, making current solutions impractical without a more strategic approach.

Workload Allocation Through Constraint Optimization

To address this, Stream implements WACO, a workload allocator and scheduler which uses constraint optimization to only allocate and schedule the *steady-state* portion of the workload graph. This portion comprises the work within each layer that is repeated across numerous iterations of the layer-fused process. Fig. 10 highlights its building blocks. The workload graph is divided into one or more *layer stacks* (Section V-A) of which the steady state is extracted (Section V-B). Furthermore, partitioning the CTs across cores, referred to as *CT parallelism*, provides WACO additional optimization capabilities (Section V-C). A constraint optimization formulation accounts for potential overlap between steady-state iterations to efficiently interleave them (Section V-D).

A. Layer Stacks Identification

The CT workload graph is separated into multiple layer stacks. Within one layer stack, the execution of CTs will be fused. In this work, a stack is limited to a contiguous set of

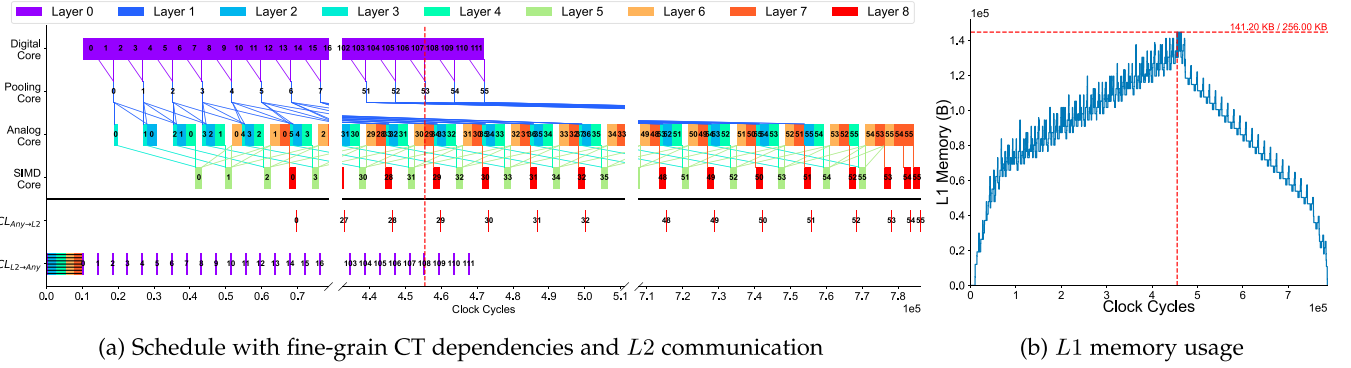


Fig. 9. (a) Layer-fused schedule. (b) L1 memory usage of ResNet18 validation on DIANA.

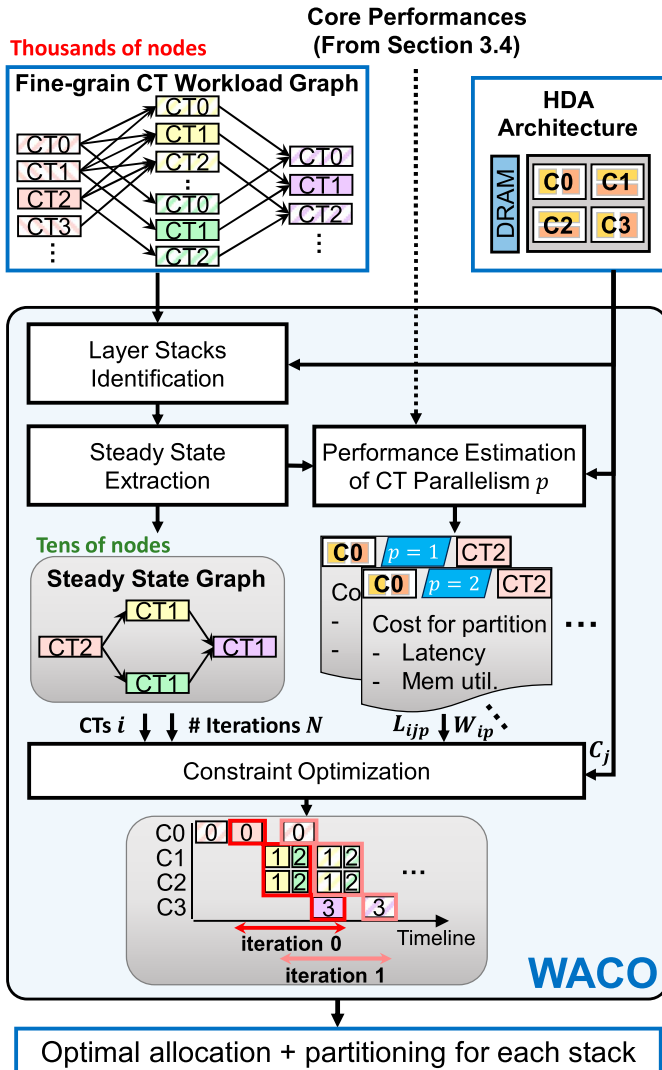


Fig. 10. The workload allocation is optimized using constraint optimization of the steady-state CTs with the cost of different degrees of CT parallelism in the K dimension.

layers whose combined weights don't exceed the HDA cores' memory capacity. This ensures weights of each layer can remain stationary on-chip throughout the layer-fused execution of the stack.

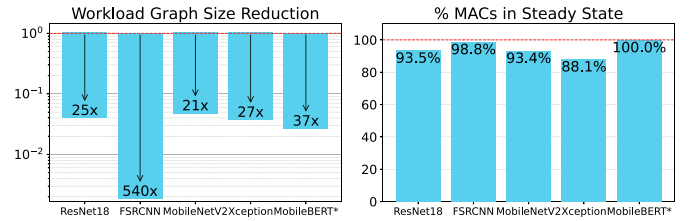


Fig. 11. The steady-state portion of different DNN workloads greatly reduces graph size while representing a large percentage of total workload MACs.

B. Steady State Extraction

For each stack, the steady-state portion is extracted by iterating through the CTs of the last layer in the stack. For each iteration, the unique predecessors of the last layer's CTs are retrieved and counted. These predecessors are the CTs of each layer in the stack required to process the current sink CT that have not been computed in a previous iteration. This is achieved using constant-time hash table lookups. If two sink CTs have the same work and their count is increased. The final counts are weighed by the number of MAC operations to find the iteration that is most significant. The CTs of this iteration are called the *steady state* and used for the subsequent steps. Fig. 11 demonstrates that considering only the steady state CTs greatly reduces the size of the graph for line-based layer fusion, while still representing a large portion of the MAC operations of the entire workload. The greatest reduction is seen for FSRCNN, which can be separated into 540 iterations of only 8 CTs thanks to its simple network structure and 540×540 input activation.

C. Allocating Computation Tiles to Multiple Cores

WACO enables a CT to be further parallelized across multiple cores in the output channel dimension K . This ensures that the weights—which are reused throughout the entire fused stack—remain distinct across cores. This parallelism factor, denoted with p , lies between 1 and the maximum number of cores the CT can be allocated to and is required to be a divisor of K . To achieve an optimal allocation across cores while respecting constraints, it is necessary to calculate the weight requirement

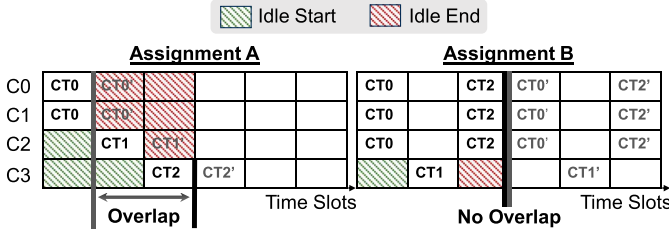


Fig. 12. Illustration of two different assignments of three steady-state tiles on four cores, leading to different overlap due to the idle start and end slots mismatch between cores.

and estimate the performance of each partition for the possible p values. The weight requirement for tile i with parallelization factor p is computed as:

$$W_{ip} = \frac{W_i}{p}$$

For latency, the CT parallelization across cores can have an unexpected performance impact in HDAs, as 1) the latency can be different on different cores, and 2) some cores might require parallelization of K within the core, thus causing array underutilization when parallelizing across many cores. These effects are automatically accounted for by WACO: the latency L_{ijp} of each CT i on each core j with parallelization factor p is reduced by the same factor p as long as the mapping onto the core leaves enough temporal loops K_{ij}^t (the loops that remain after taking the within-core spatial parallelization into account). If there are not enough temporal loops, the latency reduction is limited. More formally:

$$L_{ijp} = \begin{cases} \frac{L_{ij}}{p} & \text{if } p \leq K_{ij}^t \\ \frac{L_{ij}}{K_{ij}^t} & \text{if } p > K_{ij}^t \end{cases}$$

D. Constraint Optimization

The core of WACO's optimization is captured through a series of variables and constraints, formulated to delineate the scheduling and allocation of the steady-state CTs. This problem is formulated and solved using Gurobi [35] within the Stream framework.

1) *Objective Function*: The objective of the optimization is to minimize the total latency lat across N steady-state iterations, considering $(N - 1)$ overlaps o between steady-state iterations, as illustrated in Figs. 10 and 12.

2) *Constants*: The constants provided to the constraint formulation are:

- W_{ip} : The number of weights required for tile i given a CT parallelism factor of p .
- C_j : The weight memory capacity of core j .
- $D_{i,i'}$: The dependencies between all tiles i and i' .
- L_{ijp} : The latency of tile i on core j given a CT parallelism factor of p .

3) *Variables*: The key base variables in the formulation are:

- a_{ijs} : Represents the binary assignment of tile i to core j during time slot s . The latency of a time slot is not fixed and depends on the assignments. These form the basis of both the allocation and scheduling processes.

- p_i : A one-hot binary vector of dimension K_i indicating the chosen CT parallelism for tile i . It ensures that only one parallelism choice is selected for each tile.
- $IdleStart_{js}$ and $IdleEnd_{js}$: Binary variables that indicate for each timeslot s and each core j if the core is idle before any CT has been scheduled ($IdleStart$) or if the core is idle after all CTs have been scheduled ($IdleEnd$).
- Other variables are integer and derived from the base variables in the constraints below.

4) *Assignment, Dependency & Weight Constraints*: The constraints are defined as follows:

Assignment Constraints To ensure that no more than one tile is assigned to any core within a specific time slot, we impose the following constraint:

$$\sum_i a_{ijs} \leq 1, \quad \forall j, \forall s \quad (1)$$

For the parallelism vector p_i , we enforce that only one parallelism option is selected:

$$\sum_k p_{ik} = 1 \quad \forall i \quad (2)$$

The total number of assignments across cores and slots for a tile i must match the selected parallelism, defined as:

$$\sum_{j,s} a_{ijs} = \sum_k k \cdot p_{ik} \quad \forall i \quad (3)$$

Slot and Dependency Constraints The slot $Slot_i$ assigned to each tile i must respect the dependency constraints defined by the dependency matrix D :

$$Slot_i \leq Slot_{i'} - 1 \quad \forall (i, i') \in D \quad (4)$$

Weight Constraints The weights stored on each core j are quantified and constrained to not exceed the weight memory capacity of each core:

$$w_j = \sum_i \left(\sum_k (W_{ik} \cdot p_{ik}) \cdot \sum_s a_{ijs} \right) \leq C_j \quad \forall j \quad (5)$$

5) *Latency Constraints*: To accurately model the latency based on the assignments a , we define the following variables:

- y_j : Represents the total number of assignments on each core:

$$y_j = \sum_{i,s} a_{ijs} \quad \forall j \quad (6)$$

- z_{js}^{incl} and z_{js}^{excl} : Represent the inclusive and exclusive sum of assignments on core j up to and including, or excluding, slot s , respectively:

$$z_{js}^{incl} = \sum_{z=0}^s \sum_i a_{ijz}; \quad z_{js}^{excl} = \sum_{z=0}^{s-1} \sum_i a_{ijz} \quad \forall j, \forall s \quad (7)$$

- l_{ij} : Represents the latency of tile i on core j , calculated by selecting the appropriate latency L_{ijk} based on the chosen parallelism:

$$l_{ij} = \sum_{s,k} a_{ijs} \cdot p_{ik} \cdot L_{ijk}, \quad \forall i, \forall j \quad (8)$$

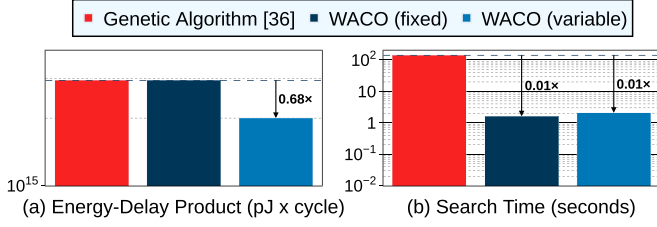


Fig. 13. Comparison of (a) energy-delay product and (b) required search time for the allocation and scheduling of ResNet18 on a quad-core architecture with four WS cores, using a state-of-the-art genetic algorithm-based optimizer and WACO (see Table III).

- l_s : Represents the maximal latency for any tile assigned to slot s across all cores:

$$l_s = \max_j \left(\sum_i l_{ij} \cdot a_{ijs} \right) \quad \forall s \quad (9)$$

- $idle_j$: Represents the total start and end idle time on each core, as illustrated in Fig. 12:

$$idle_j = \sum_s (IdleStart_{js} + IdleEnd_{js}) \cdot l_s \quad \forall j \quad (10)$$

- o : Represents the achievable overlap between steady-state iterations, defined as the minimum idle time across all cores:

$$o = \min_j (idle_j) \quad (11)$$

- lat : The total latency is computed as:

$$lat = N \cdot \sum_s l_s - (N - 1) \cdot o \quad (12)$$

E. Comparison With SotA

Fig. 13 provides a comparative analysis of the Energy-Delay Product (EDP) and search time for optimizing the allocation and scheduling of ResNet18 on a quad-core WS HDA. The analysis contrasts a state-of-the-art Genetic Algorithm (GA)-based search [36] with two configurations of WACO: one fixes $p = 1$ to ensure a direct comparison with the GA approach, which does not support CT parallelization across cores, and another allows variable CT parallelism to explore the benefits of distributing a CT across multiple cores to enhance utilization. WACO with $p = 1$ achieves similar EDP compared to the GA while significantly reducing the search time. Additionally, enabling CT parallelization offers greater efficiency at only a marginal search time increase, reducing the EDP by 32% by filling idle slots present due to workload imbalances.

VI. EXPLORATION

In this section, we deploy Stream to co-explore the optimal allocation in combination with architectural decisions across different workloads in two exploration studies. The studies are repeated for five DNN workloads: ResNet18 [37], FSRCNN [34], MobileNetV2 [38], Xception [39] and the first body of MobileBERT [40]. The framework is evaluated offline, with

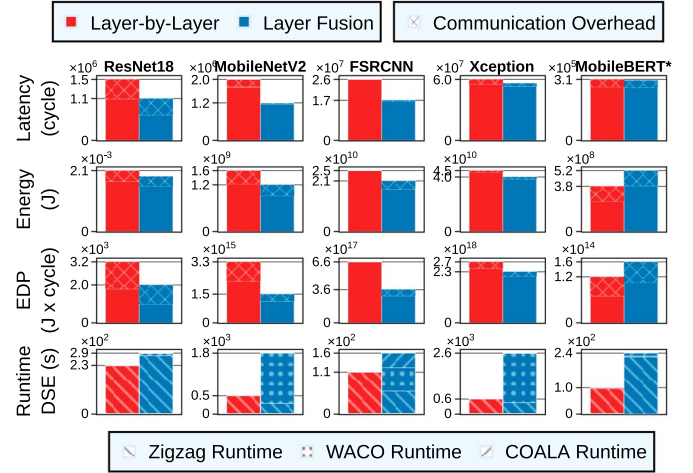


Fig. 14. Layer-by-layer vs. layer fusion performance of WACO-optimized allocation for a quad-core HDA with four WS cores as in Table III. * Exploration of first transformer block.

parallel processing of different input combinations: workload, HDA and scheduling paradigm.

The HDA architectures used in the different studies are built from two core architectures: a weight and an output stationary accelerator core. The core architectures are shown in Fig. 2, with the specific PE array shape, dataflow, memory capacities and memory bandwidths specified in Table III. All memory read costs and write costs are extracted in 22 nm using CACTI 7 [41], an open-source memory profiling tool. Each PE includes a 4 B register file. The output register file of each WS column is 128 B. The output local buffer of the OS core is matched in capacity (128 B times the number of columns) and bandwidth (read-out of one row at 16 bit precision as opposed to writing of two rows at 8 bit in WS).

The cores are interconnected through a core-to-core communication bus of 256 bits per clock cycle. A single port of 128 bits per clock cycle models contention of off-chip accesses to a 256 MiB DRAM memory.

For all experiments, WACO optimizes the allocation with variable CT parallelism, which is then evaluated by COALAt to obtain latency and energy efficiency estimations.

A. Workload Diversity & Scheduling Granularity

First, the benefits of layer fusion compared to layer-by-layer scheduling are analyzed for the five workloads.

Fig. 14 compares the latency, energy, EDP and DSE runtime of the best found allocation by WACO onto a quad-core HDA with four WS cores. The highlighted sections denote the communication overheads. MobileNetV2 and FSRCNN benefit largely from layer fusion, as they are activation-dominant. Their EDP is reduced by 2.2x and 1.8x, respectively, under layer fusion, thanks to the small weight size which allows WACO to fuse many layers into a single stack, bringing large benefits as all intermediate activation can be stored on-chip. MobileBERT challenges the current fusion strategy due to the complexity of its attention mechanism and reduced energy efficiency during

TABLE III
HDA ARCHITECTURES FOR THE DATAFLOW HETEROGENEITY STUDY. THE ARCHITECTURE IS VARIED FROM 1 TO 8 CORES WITH A TOTAL OF 2^{12} PES AND 4 MiB OF ON-CHIP MEMORY

HDA Arch.	No. Arch.	Weight Stationary Core Architecture Fig. 2(b)								Output Stationary Core Architecture Fig. 2(c)							
		PE Array row $\times$ col	Dataflow				Act Mem		W Mem		PE Array row $\times$ col	Dataflow		Act Mem		W Mem	
			C	FX	FY	K	Size MiB	BW B	Size MiB	BW B		OX	K	Size MiB	BW B	Size MiB	BW B
1 Core	2	72×64	8	3	3	64	2	72	2	128	64×64	64	64	2	64	2	64
2 Core	3	36×64	4	3	3	64	1	36	1	128	32×64	32	64	1	32	1	64
4 Core	5	36×32	4	3	3	32	0.5	36	0.5	64	32×32	32	32	0.5	32	0.5	32
8 Core	9	18×32	2	3	3	32	0.25	18	0.25	64	16×32	16	32	0.25	16	0.25	32

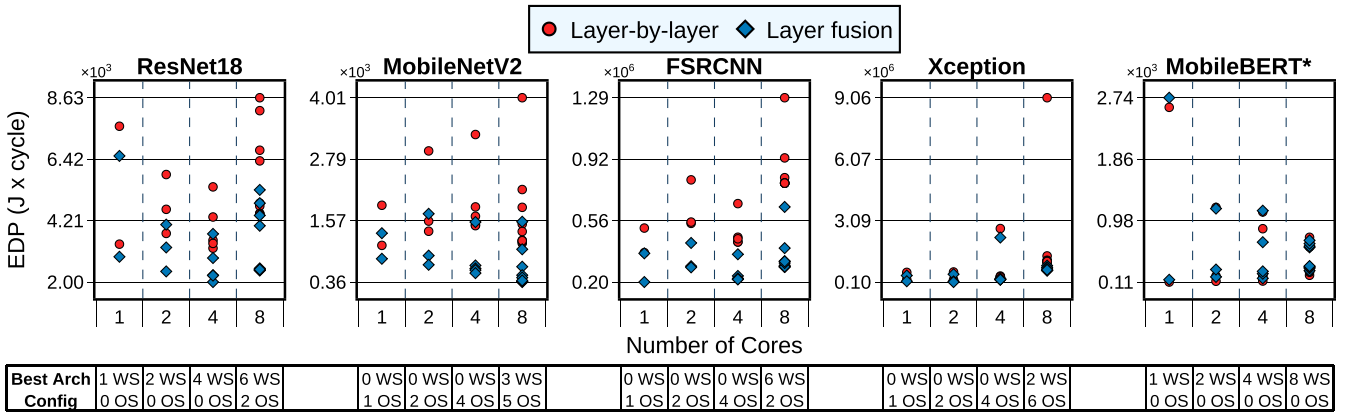


Fig. 15. Exploration results showing EDP impact of varying scheduling, number of cores and dataflow heterogeneity within a fixed area of 2^{12} PES and 4 MiB.

layer fusion, caused by limited weight reuse in dense matrix multiplication layers. Stream offers potential for exploring new fusion strategies and specialized dataflows to address these issues, making it a viable option for future research.

B. Dataflow Heterogeneity

The initial exploration maintained a uniform dataflow across all four cores, resulting in a homogeneous architecture. This demonstrated that layer fusion is advantageous for various workloads; however, the system's performance also significantly depends on the number of cores and the specific dataflow employed in each core. Consequently, a subsequent study evaluates the performance implications of varying both the number of cores and their dataflows. This study explores a range of architectures from a single core up to eight cores, all constrained to the same total number of processing elements (PEs) and memory capacity, thereby maintaining an iso-area condition. For each core count, the study examines $\binom{n}{k}$ possible combinations of architecture configurations, where $n = 2$ represents the two types of cores available, and k is the number of cores in the system.

Fig. 15 presents a comparison of Energy-Delay Product (EDP) performances for different workloads under layer-by-layer and layer fusion strategies across various core counts

and dataflow configurations. The optimal architecture for each scenario is shown below, revealing that distinct workloads exhibit preferences for specific dataflows. Notably, MobileNetV2, characterized by numerous small layers, shows improved performance with a greater number of OS cores, which are more effective for depthwise convolutional layers. Conversely, other networks experience diminished performance in larger core counts due to the increased overhead associated with core-to-core communication.

C. Hardware Scale-Up

Finally, the hardware is scaled to larger sizes to evaluate the interaction between the available compute elements and the optimal number of cores by increasing the PE budget from 2^{12} to 2^{14} and 2^{16} . The dataflows are scaled similarly to Table III, keeping the array as square as possible. While for smaller PE budgets, a single-core system may achieve the best utilization, this can change as the array size increases, particularly if the layers do not exhibit sufficient parallelism to fully leverage the expanded array anymore.

Fig. 16 summarizes the results of the scaled out simulations across varying compute budgets. For each PE budget, the EDP-optimal architecture configuration—comprising core counts and dataflow per core—is presented across various workloads

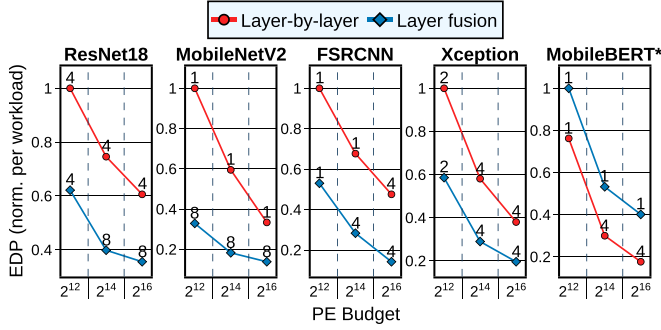


Fig. 16. EDP performance improvements of the hardware scale-up exploration. The annotations show increasing optimal core counts for larger budgets.

and scheduling paradigms. The annotated number represents the optimal core count. The observed increase in optimal core counts as PE budgets expand highlights the strong relationship between architecture configuration, workload type, and compute resources.

VII. CONCLUSION

This work presented *Stream*, a design space framework for the optimization of layer-fused deep neural networks on heterogeneous dataflow accelerators, addressing the challenges posed by ever-growing network sizes and the need for reduced latency and energy consumption at the edge.

Central to *Stream*'s implementation is a communication and memory-aware latency analysis (COALA), which enables rapid performance estimation in terms of latency and energy efficiency. COALA is validated against three state-of-the-art accelerator systems that employ layer fusion. A workload allocation engine (WACO) strategically allocates computational cores to maximize efficiency. WACO leverages constraint optimization techniques that, in combination with the parallelism and memory efficiency afforded by layer fusion, significantly enhance hardware utilization. This demonstrated a reduction of up to $2.2\times$ in energy-delay product compared to traditional layer-by-layer scheduling.

Moreover, the flexibility of *Stream* offers a comprehensive tool for future research and development in efficient DNN deployment on evolving accelerator architectures.

ACKNOWLEDGMENT

The authors thank Koen Goetschalckx for his valuable comments.

REFERENCES

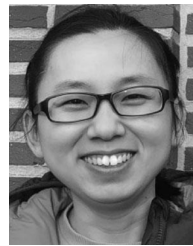
- [1] Z. Du et al., "ShiDianNao: Shifting vision processing closer to the sensor," in *Proc. ACM/IEEE 42nd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2015, pp. 92–104.
- [2] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [3] B. Moons, R. Uytterhoeven, W. Dehaene, and M. Verhelst, "14.5 enVision: A 0.26-to-10TOPS/W subword-parallel dynamic-voltage-accuracy-frequency-scalable convolutional neural network processor in 28nm

- FDSOI," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA: Piscataway, NJ, USA: IEEE Press, 2017, pp. 246–247, doi: 10.1109/ISSCC.2017.7870353.
- [4] S. Ghodrati et al., "Planaria: Dynamic architecture fission for spatial multi-tenant acceleration of deep neural networks," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2020, pp. 681–697.
- [5] Y. S. Shao et al., "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO '52)*, New York, NY, USA: ACM, 2019, pp. 14–27, doi: 10.1145/3352460.3358302.
- [6] R. Radway et al., "Illusion of large on-chip memory by networked computing chips for neural network inference," *Nature Electron.*, vol. 4, pp. 71–80, Jan. 2021.
- [7] H. Jia et al., "Scalable and programmable neural network inference accelerator based on in-memory computing," *IEEE J. Solid-State Circuits*, vol. 57, no. 1, pp. 198–211, Jan. 2022.
- [8] D. Shin, J. Lee, J. Lee, J. Lee, and H.-J. Yoo, "DNPU: An energy-efficient deep-learning processor with heterogeneous multi-core architecture," *IEEE Micro*, vol. 38, no. 5, pp. 85–93, Sep./Oct. 2018.
- [9] H. Kwon, L. Lai, M. Pellauer, T. Krishna, Y. Hsin Chen, and V. Chandra, "Heterogeneous dataflow accelerators for multi-DNN workloads," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2021, pp. 71–83.
- [10] K. Ueyoshi et al., "DIANA: An end-to-end energy-efficient digital analog hybrid neural network SoC," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, 2022, pp. 1–3.
- [11] A. Garofalo et al., "A heterogeneous in-memory computing cluster for flexible end-to-end inference of real-world deep neural networks," 2022, *arXiv:2201.01089*.
- [12] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer CNN accelerators," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2016, pp. 1–12.
- [13] K. Goetschalckx and M. Verhelst, "Breaking high-resolution CNN bandwidth barriers with enhanced depth-first execution," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 323–331, Jun. 2019.
- [14] TVM RFCS, "Arm@ Ethos™-U cascading scheduler," GitHub. Accessed: Apr. 12, 2024. [Online]. Available: <https://github.com/apache/tvm-rfcs/blob/main/rfcs/0037-arm-ethosu-cascading-scheduler.md>
- [15] K. Goetschalckx and M. Verhelst, "DepFiN: A 12nm, 3.8 TOPs depth-first CNN processor for high resolution image processing," in *Proc. Symp. VLSI Circuits*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1–2.
- [16] Z. Liu et al., "DLFusion: An auto-tuning compiler for layer fusion on deep neural network accelerator," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Social Comput. Netw. (ISPA/BDCloud/SocialCom/SustainCom)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 118–127.
- [17] S. Colleman and M. Verhelst, "High-utilization, high-flexibility depth-first CNN coprocessor for image pixel processing on FPGA," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 29, no. 3, pp. 461–471, Mar. 2021.
- [18] L. Mei, K. Goetschalckx, A. Symons, and M. Verhelst, "DeFiNES: Enabling fast exploration of the depth-first scheduling space for DNN accelerators through analytical modeling," 2022, *arXiv:2212.05344*.
- [19] Y. Xing et al., "DNNVM: End-to-end compiler leveraging operation fusion FPGA-based CNN accelerators," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA '19)*, New York, NY, USA: ACM, 2019, pp. 187–188, doi: 10.1145/3289602.3293972.
- [20] X. Cai, Y. Wang, K. Tu, C. Gao, and L. Zhang, "Olympus: Reaching memory-optimality on DNN processors," *IEEE Trans. Comput.*, vol. 71, no. 8, pp. 1939–1951, Aug. 2022.
- [21] S. Zheng et al., "TileFlow: A framework for modeling fusion dataflow via tree-based analysis," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2023, pp. 1271–1288.
- [22] M. Verhelst and B. Moons, "Embedded deep neural network processing: Algorithmic and processor techniques bring deep learning to IoT and edge devices," *IEEE Solid-State Circuits Mag.*, vol. 9, no. 4, pp. 55–65, Fall 2017.
- [23] A. Symons, L. Mei, and M. Verhelst, "LOMA: Fast auto-scheduling on DNN accelerators through loop-order-based memory allocation," in *Proc. IEEE 3rd Int. Conf. Artif. Intell. Circuits Syst. (AICAS)*, 2021, pp. 1–4.
- [24] X. Yang et al., *Interstellar: Using Halide's Scheduling Language to Analyze DNN Accelerators*. New York, NY, USA: ACM, 2020, pp. 369–383, doi: 10.1145/3373376.3378514.

- [25] H. Kwon, P. Chatarasi, V. Sarkar, T. Krishna, M. Pellauer, and A. Parashar, "MAESTRO: A data-centric approach to understand reuse, performance, and hardware cost of DNN mappings," *IEEE Micro*, vol. 40, no. 3, pp. 20–29, May/Jun. 2020.
- [26] A. Parashar et al., "Timeloop: A systematic approach to DNN accelerator evaluation," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2019, pp. 304–315.
- [27] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst, "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1160–1174, Aug. 2021.
- [28] Q. Huang et al., "CoSA: Scheduling by constrained optimization for spatial accelerators," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2021, pp. 554–566.
- [29] K. Hegde, P.-A. Tsai, S. Huang, V. Chandra, A. Parashar, and C. W. Fletcher, "Mind mappings: Enabling efficient algorithm-accelerator mapping space search," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Operating Syst. (ASPLOS)*. New York, NY, USA: ACM, 2021, pp. 943–958, doi: 10.1145/3445814.3446762.
- [30] S.-C. Kao and T. Krishna, "GAMMA: Automating the HW mapping of DNN models on accelerators via genetic algorithm," in *Proc. 39th Int. Conf. Comput.-Aided Des. (ICCAD)*, 2020, pp. 1–9.
- [31] J. Bai et al., "ONNX: Open neural network exchange," GitHub. [Online]. Available: <https://github.com/onnx/onnx>
- [32] A. Guttman, "R-Trees: A dynamic index structure for spatial searching," in *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD '84)*, New York, NY, USA: ACM, 1984, pp. 47–57, doi: 10.1145/602259.602266.
- [33] L. Mei, H. Liu, T. Wu, H. E. Sumbul, M. Verhelst, and E. Beigne, "A uniform latency model for DNN accelerators with diverse architectures and dataflows," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, 2022, pp. 220–225.
- [34] C. Dong, C. C. Loy, and X. Tang, "Accelerating the super-resolution convolutional neural network," in *Proc. Eur. Conf. Comput. Vis.*, Amsterdam, The Netherlands: Springer, 2016, pp. 391–407.
- [35] "Gurobi Optimizer Reference Manual," Gurobi Optimization, LLC, 2023. [Online]. Available: <https://www.gurobi.com>
- [36] S. Karl, A. Symons, N. Fafous, and M. Verhelst, "Genetic algorithm-based framework for layer-fused scheduling of multiple DNNs on multi-core systems," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, 2023, pp. 1–6.
- [37] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [38] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.
- [39] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1251–1258.
- [40] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: A compact task-agnostic BERT for resource-limited devices," 2020, *arXiv:2004.02984*.
- [41] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optimization*, vol. 14, no. 2, pp. 1–25, 2017.



Arne Symons received the B.Sc. and M.Sc. degrees in electrical engineering from KU Leuven, Leuven, Belgium, in 2018 and 2020, respectively. He is currently working toward the Ph.D. degree in mapping optimization of machine learning workloads on specialized accelerator architectures. In 2022, he was a Visiting Researcher with Robust Systems Group, Stanford University, and interned as a Research Scientist with Meta in 2023.



Linyan Mei received the B.Sc. degree in electronic design and technology from Beijing Institute of Technology (BIT), Beijing, China, in 2016, and the M.Sc. and Ph.D. degrees in electrical engineering from KU Leuven, Leuven, Belgium, in 2018 and 2023, respectively. Her Ph.D. thesis is entitled *Design Space Exploration of Deep Learning Accelerators*. She was an intern with IMEC, Leuven, Belgium, from 2017 to 2018 and with Meta in 2021. She is currently working at Huawei, China on hardware–software co-design of machine learning

workloads.



Steven Coleman received the B.Sc., M.Sc., and Ph.D. degrees in electrical engineering from KU Leuven, Leuven, Belgium, in 2016, 2018, and 2024, respectively. His Ph.D. thesis is entitled *Modeling and Analysis for Efficient Hardware Mapping of Neural Network Algorithms*. He is currently working with Axelera AI in Leuven, Belgium.



Pouya Houshmand (Member, IEEE) received the B.Sc. and M.Sc. degrees in electrical engineering from the Polytechnic of Turin, Italy, in 2017 and 2019, respectively. He is currently working toward the Ph.D. degree in architectures for deep neural network accelerators with the ESAT-MICAS Laboratories, KU Leuven, Leuven, Belgium. His research interests include algorithm–hardware co-design, in-memory computing, and emerging technologies.



Sebastian Karl received the B.Sc. and M.Sc. degrees in electrical engineering from the Technical University of Munich, Germany, in 2020 and 2022, respectively, and the M.Sc. Management degree from Lund University School of Economics and Management, in 2023. He is currently working as a Project Manager and a Business Developer for the business field neuromorphic computing with Fraunhofer IIS, Erlangen, Germany.



Marian Verhelst (Fellow, IEEE) received the Ph.D. degree from KU Leuven, Leuven, Belgium, in 2008. She is a Full-Time Professor with ESAT-MICAS Laboratories, KU Leuven. Her research focuses on embedded machine learning, hardware accelerators, and low-power edge processing. Before that, she worked with the Intel Labs, US, from 2008 to 2011. She is an SSCS Distinguished Lecturer, a member of the Young Academy of Belgium, an Associate Editor of IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION SYSTEMS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS II: EXPRESS BRIEFS, and IEEE JOURNAL OF SOLID-STATE CIRCUITS. She currently holds an ERC Grant from the European Union and was the laureate of the Royal Academy of Belgium.